# HBF-Sim: An Extensible HBF Simulator for Large-scale GPU Memory Systems

Yaqi Li[1,2], Jing Wang[1], Junfeng Wang[1], Long Yang[1,2], Han Yan[1], Xiaohu Chai[1], Liang Shi*[1,2]

[1]*East China Normal University* [2]*Shanghai Innovation Institute*

## Abstract

High-bandwidth flash (HBF) is introduced to address the memory wall, which can co-package a dense NAND stack with the GPU, targeting the performance gap between near-accelerator bandwidth and flash density. HBF, however, is neither a large HBM nor a fast NVMe SSD. Its usable bandwidth depends on how GPU cache-line requests map onto NAND pages, how concurrency spreads across channel-affine die sets, and how media management interacts with the GPU memory pipeline. To our knowledge, existing GPU, SSD, or HBF simulators cannot faithfully model this behavior.

We present HBF-Sim, an extensible, reusable, and faithful HBF simulator integrated with simulated GPUs. It closes the loop between GPU issue limits, device queuing, and NAND behavior in one end-to-end request path. HBF-Sim separates a GPU–HBF interaction controller from page-based parallel stack storage, and it models the full GPU–HBF request path. It provides an MSHR-based address mapping table that merges cache-line requests into page-based operations, as well as a page-based multi-stack flash manager for highly parallel reads and writes. Validation tests and device-level microbenchmarks expose performance bottlenecks caused by limited channel distribution and resource conflicts, offering concrete guidance for next-generation HBF architectures.

## 1 Introduction

Memory-hungry workloads such as LLM training and agentic inference are increasingly bounded by the limited capacity of on-chip HBM and DDR memory [6]. Model weights are largely read-only and fixed in size [18], whereas the key–value (KV) cache grows with context length and batch size. During decoding, insufficient HBM capacity forces systems either to evict reusable KV entries or to shard models across more GPUs. Recent work has explored CPU DRAM and flash as extensions to GPU memory [1, 29], but these remote memory tiers remain constrained by PCIe bandwidth, access latency, and software-stack overhead.

High-bandwidth flash (HBF) offers a promising way to alleviate the GPU memory wall. As a package-local memory tier for GPUs, it provides 512 GiB of density and multi-TB/s interface bandwidth at a fraction of HBM's cost. A recent Open Compute Project (OCP) specification [24], proposed by SK hynix and SanDisk, integrates a dense NAND stack co-packaged with the GPU and served by up to 16 independent UCIe host channels, yielding an aggregate interface bandwidth of 3.072 TB/s. HBF is thus positioned as a "memory-interface flash" tier, offering near-accelerator latency and bandwidth together with flash density and cost.

However, simulating HBF behavior is not easy. An HBF-enhanced architecture is neither a large HBM nor a fast NVMe SSD. The main difficulty is to leverage the available bandwidth, which depends on several factors. In detail, it may depend on how 64 B GPU cache-line requests map onto 4 KiB NAND pages, how request concurrency spreads across channel-affine die sets, and how 4 KiB write aggregation and backpressure interact with the GPU's memory pipeline. It relates to media management, which prohibits the garbage-collection machinery that every SSD simulator assumes. Unforturenately, existing platform ignore these interactions. GPU simulators (GPGPU-Sim [3], Accel-Sim [13]) abstract off-chip storage as DRAM. SSD simulators and emulators (MQSim [32], FEMU [19], Cylon [36]) implement NVMe/CXL block-device semantics. Recent HBF studies (e.g., $H^3$ [7], HAVEN [8], TileLens [9], FlashAccel [34], DASH [15], and Li et al. [21]) build on internal or application-specific models. Their publicly available descriptions do not establish support for the full set of GPU–HBF request and completion semantics studied in this work.

This paper presents HBF-Sim, an extensible, reusable, and more faithful HBF simulator integrated with Accel-Sim/GPGPU-Sim. It models the full GPU–HBF request path and is designed to study hardware and system design. HBF-Sim explicitly separates the GPU–HBF interaction controller, which sits above the base die and owns granularity alignment, address mapping, and memory/flash management, from the page-based parallel stack storage, which owns coalescing-

based data storage and data retrieval. The GPU drives HBF through ordinary load/store interfaces, while a detailed per-request trace exposes both request state transitions and accounting. We make the following contributions:

- **An Open-source and Scalable Simulation Framework.** Our HBF-Sim mainly consists of two parts, GPU-HBF interaction controller for data alignment and page-based flash manager for multi-stack parallelism. We implement HBF-Sim as an extension to Accel-Sim with a configurable GPU-HBF request/ completion path and release it as open source (§4).
- **High-Performance Flash Access Strategies.** We propose an MSHR-based address mapping table that efficiently merges GPU cache-line requests into page-based HBF operations (§4.1). We further design a page-based, multi-stack flash management method that supports highly parallel reads and writes (§4.2).
- **Flexible System Extension and Performance Optimization.** Our simulator is built on an extensible architecture that supports flexible configuration of storage capacity, bandwidth, NAND timing, placement, mapping, and controller limits (§4). Users can readily extend this architecture and plug in their own optimizations (§6).
- **Insights for Future HBF-Enhanced System Design.** Through validation tests and device-level microbenchmarks, we measure fine-grained workload behavior and cost (§5). Our observations identify performance bottlenecks caused by limited channel distribution and resource conflicts, which inspire potential designs for next-generation HBF architectures.

## 2 Background

### 2.1 HBF Device Semantics

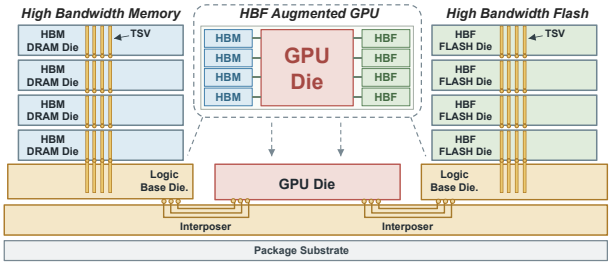



Figure 1: **Overview of an HBF-augmented GPU package.** The inset shows HBM and HBF stacks beside the GPU; cross-section shows DRAM and NAND dies, TSVs, microbumps, logic/base dies, the interposer, and package substrate.

HBF is defined as a new storage tier that bridges the gap between HBM (High Bandwidth Memory) and SSDs (Solid-State Drives), aimed at addressing the "memory wall" challenge posed by the era of large AI models. SK Hynix and SanDisk have jointly released the first High Bandwidth Flash (HBF)standard specification (version 0.7.0) [24] and made it available through the OCP (Open Compute Project). HBF is exposed to the host through a memory-like interface rather than as a block device. The cube provides up to 16 independent host channels, each bound to a fixed set of NAND dies and a local address space. Because requests on one channel cannot access resources owned by another, placement directly determines how much parallelism the device can deliver. The aggregate interface bandwidth reaches 3.072 TB/s across all channels, but that bandwidth is only attainable when requests are mapped to the appropriate channel-local resources.

| | HBM | HBF | CXL-flash | NVMe SSD |
|---|---|---|---|---|
| Interface | pseudo-ch | UCIe/AXI | CXL.mem | PCIe/NVMe |
| Granularity | 32–64B | 64B/4KiB | 64B | 512B–4KiB |
| Latency | 100 ns | 1–15 $\mu$s | $\mu$s–ms | 100 $\mu$s |
| Device GC | no | **no** | partial | yes |
| Capacity | 80–192 GB | configurable 512 GiB span | TB-class | TB-class |
| BW | 1–8 TB/s | nominal 3.07 TB/s | 10s GB/s | 3–14 GB/s |

Table 1: **Comparison of representative memory and storage tiers.** Parameters are based on respective industry specifications (JEDEC HBM3/3e, OCP HBF v0.7.0 [24], CXL 3.1 [5]) and enterprise SSD datasheets [22, 27]. HBF latency reflects representative flash media access and buffer-hit timing profiles.

HBF differs from conventional memory in its media semantics. Host reads are 64B granular, while the NAND HBFs operates at 4 KiB page granularity. A host read must therefore be aligned to a single NAND page, which may be served from a page buffer when ordering constraints permit. Writes are non-posted: the controller accumulates a full 4 KiB page before programming NAND, and completion occurs only after programming finishes. Pages within a block are written sequentially from page 0.

Unlike an SSD, there's no device-managed garbage collection on HBF. Instead, wear management is handled on the host side through zone remapping, and no background valid-page migration is performed. Thus, SSD-style FTLs serve only as a reference, not as the primary model for HBF behavior.

Table 1 compares HBF with HBM, CXL-flash, and NVMe SSDs. Compared with HBM, HBF offers much larger capacity but retains flash-like latency and page granularity. Compared with CXL-attached flash and NVMe SSDs, HBF keeps a memory-like load/store interface while moving the flash device onto the GPU package, thereby reducing host-stack and link overheads.

### 2.2 HBF in the GPU Memory Hierarchy

HBF becomes a new package-local flash tier alongside HBM on GPU, as shown in Figure 1. Its NAND dies and logic/base die connect to the GPU through package-level links, but this

| Platform | Configurable GPU timing | NAND page timing | Load/store flash access | HBF request and completion |
|---|---|---|---|---|
| GPGPU-Sim / Accel-Sim [3, 13] | Yes | No | No | No |
| MQSim [32] | No | Yes | No | No |
| SimpleSSD (gem5 CPU) [4, 11] | No | Yes | No | No |
| FEMU (BlackBox) [19] | No | Yes | No | No |
| Cylon [36] | No | Yes | Yes | No |
| **HBF-Sim (ours)** | **Yes** | **Yes** | **Yes** | **Yes** |

Table 2: **Platform capabilities for GPU–HBF studies.** Yes/No indicates support in the listed configurations.

physical proximity does not give HBF DRAM-like access behavior. As Table 1 summarizes, HBF combines a memory-style interface with page-buffered reads, channel-affine media resources, and non-posted page writes. Modeling its place in the GPU memory hierarchy therefore requires both GPU execution timing and these flash semantics.

Accel-Sim and GPGPU-Sim already model instruction execution, caches, interconnects, and warp timing [3, 13]. HBF-Sim extends this path with HBF page, channel, and completion state while retaining the ordinary DRAM path. GPU cache structures and HBF page-tracking structures have distinct roles; the HBF cache-routing policy is described in Section 4.

Consequently, studying HBF in this hierarchy requires a **closed timing loop**: GPU execution generates memory requests, HBF queues and services them under page and channel constraints, and their completion timing dictates when dependent GPU instructions can proceed. Capturing this bidirectional feedback is essential for understanding HBF performance, exposing critical limitations in existing simulation tools (§3.1).

# 3 Motivation

## 3.1 Limitations of Existing Simulators

Table 2 compares the capabilities of related works about GPU–HBF architectures. GPU timing covers instruction execution, caches, and request issue. NAND timing covers page reads and programs, while load/store flash access provides a memory-style interface. HBF simulation must connect these capabilities so that page service affects GPU execution, including write acknowledgements after programming.

GPGPU-Sim and Accel-Sim [3, 13] model GPU execution and memory timing, but their base configurations lack NAND page operations. Increasing DRAM capacity and latency does not add page-buffer reuse or page-level coalescing. MQSim [32], SimpleSSD [11], and FEMU [19] provide flash-device timing. SimpleSSD integrates with a simulated CPU, while FEMU emulates a device for guest software. Cylon [36] supports load/store access to CXL-attached flash. The listed configurations do not connect simulated GPU requests to HBF page service and return completion dependencies to GPU execution.

| class | pattern | size | modeling focus |
|---|---|---|---|
| weights | seq., read-only | 100s GB | bandwidth |
| precomputed KV | reuse reads | 100s GB–TB | page reuse |
| ephemeral KV | append-writes | 10s GB | write ordering |
| RAG vectors | random gathers | 100s GB | read amplification |

Table 3: **LLM access patterns and HBF modeling focuses.** These categories guide the configuration of our evaluation workloads.

A controller policy can change both the time needed to serve requests and when subsequent GPU requests reach HBF. For example, merging reads to the same page reduces the number of outstanding page operations. Under admission pressure, this can free controller capacity sooner and let waiting GPU requests proceed earlier. A replay with fixed arrival times keeps the original request schedule and misses this feedback, potentially underestimating the benefit of merging. Evaluating such policies therefore requires GPU request timing to respond to HBF service. Section 5.2 compares fixed-arrival replay with coupled GPU execution to demonstrate this effect.

**In summary,** A GPU–HBF simulator needs both page-level media behavior and completion feedback to GPU execution. This requirement motivates the mapping, read, write, and tracing mechanisms in Section 4.

## 3.2 Various R/W Patterns of HBF

HBF must serve different access patterns within the same system. Table 3 summarizes representative data classes in LLM and AI workloads. Sequential weight reads require sustained data delivery. Repeated accesses to KV data, such as shared prefixes [12], make page reuse relevant, while appending KV data requires write aggregation and completion tracking. Scattered RAG-vector accesses can fetch a full NAND page for a small useful region, increasing read amplification.

These patterns place different demands on the controller. Requests to the same page can share a page read or reuse buffered data. Requests to different pages need enough independent media resources to execute in parallel. When reads

and writes overlap, a long page program can delay reads that need the same resources. A policy that helps one access pattern may therefore provide little benefit for another. Studying these effects requires control over address mapping, request coalescing, and media scheduling.

We use controlled request streams to vary concurrency, locality, and write pressure separately. Keeping the request and completion semantics consistent allows us to compare policies under the same workload. Request traces connect GPU accesses to queueing, page service, and completion, helping explain whether a change reduces NAND work or shortens GPU execution. These representative categories guide the synthesis of our microbenchmarks, allowing us to isolate and study specific architectural mechanisms under controlled traffic conditions.

**In summary,** A GPU–HBF simulator needs configurable mapping, read, and write policies under a common timing model. This requirement motivates an extensible design that supports different access patterns and exposes their effects on both media activity and GPU execution.

# 4 HBF-Sim Simulator Design

HBF-Sim models two cooperating layers that determine HBF behavior: (i) the *GPU–HBF interaction controller*, which sits above the base die and governs the load/store contract between the GPU and HBF, and (ii) the *page-based parallel stack storage*, which organizes data placement, coalescing, and media access inside the HBF stacks. The former owns granularity alignment, address mapping, memory management, and flash management. The latter owns coalescing-based data storage and data retrieval. Figure 2 illustrates these functions in the GPU–HBF organization. Its upper view shows the GPU memory paths and the connections to independent HBF stacks. The expanded stack below separates request-tracking structures from data buffers and channel-owned NAND resources. Purple annotations identify the mapping, aggregation, and scheduling choices implemented on those structures.

HBF-Sim places an explicit boundary between GPU timing and HBF media timing. The GPU produces sector or cacheline requests and consumes completion events through load/store interfaces; the HBF system owns page, channel, and media state. GPU requests, HBF pages, and NAND resources are separated so that GPU timing, interface queueing, and media service remain distinguishable.

## 4.1 GPU–HBF Interaction Controller

The interaction controller translates GPU accesses into HBF page operations. It identifies the page containing each request and assigns the request to a stack, channel, and subarray. Each request retains its source GPU partition so that the completion response returns to the correct requester.

### 4.1.1 Cacheline-to-page Granularity Alignment

GPU requests must be aligned to the page granularity used by HBF. The GPU emits 32 B sectors within 64 or 128 B cache-line transactions, while HBF reads and programs 4 KiB pages. The controller groups requests by page so the read and write modules can combine accesses to the same page. Let $B$ be the start of the configured HBF address range, $a$ a request address in that range, and $P = 4096$ B the page size. We compute the zero-based logical page number as

$$p(a) = \left\lfloor \frac{a-B}{P} \right\rfloor .$$

Thus, $a-B$ is a byte offset within the HBF range. For example, accesses at $B+64$ and $B+128$ both belong to page 0, while $B+4096$ belongs to page 1.

### 4.1.2 Channel-oriented Address Mapping

A logical HBF system contains one or more independent stacks. Each stack exposes 1, 2, 4, 8, or 16 channels in the standard profiles. Configurations beyond 16 channels are supported as experimental extensions. GPU memory partitions do not create additional copies of a stack or multiply its bandwidth. The source subpartition is retained for return routing only. Total capacity is divided evenly into erase-block-aligned stack-local capacities. Page interleave or contiguous capacity slices select a stack. Stack-local addresses are restored before completion returns to the GPU. After admission, each request is assigned to one stack, one channel, and its fixed subarray resources. There is no cross-channel resource sharing.

We implement three placement policies. Page interleave distributes HBF-relative pages across channels. Contiguous placement assigns regions to selected channels. An explicit mapping table can place data classes such as weights and KV pages. The grouped placement evaluated in Section 6.1 uses this hook to rotate after every two adjacent pages. The mapping uses the address relative to the configured HBF base address. An abstract per-channel bandwidth credit models the host link. We limit outstanding operations per channel to model queue backpressure. The host-link abstraction is discussed in Section 7.

## 4.2 Page-Based Multi-Stack Flash Manager

We design the flash manager to use page-level parallelism across the channels and subarrays of each stack. The write module collects GPU stores into full pages. The read module combines requests to the same page and reuses buffered data when ordering permits. Both modules operate on 4 KiB NAND pages.

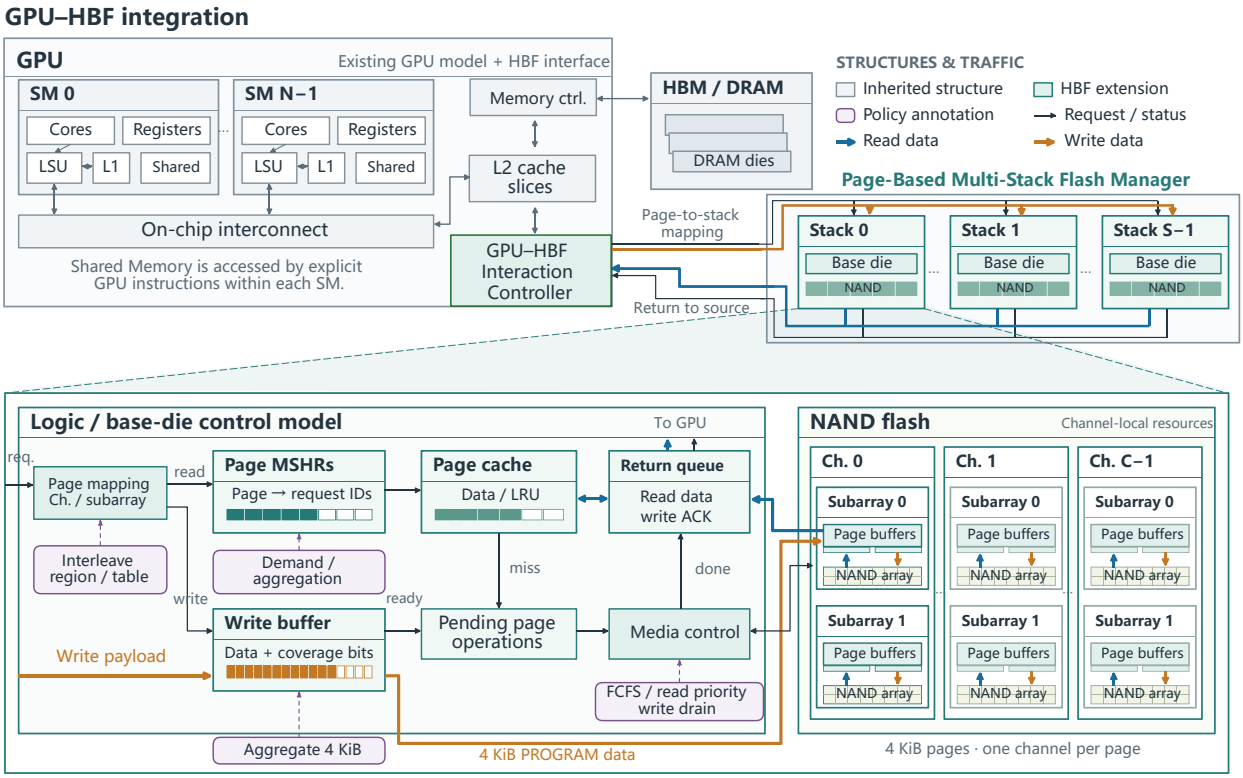



Figure 2: **HBF-Sim architecture.** The upper part shows GPU memory paths and independent HBF stacks. The lower part expands one stack into its control structures and channel-owned NAND resources. Rectangles denote modeled structures, while the figure's policy annotations identify configurable methods. Thin arrows carry requests or control/status. Thick blue and orange arrows carry read and write data.

### 4.2.1 Merge-based Write Module

We use a per-page write buffer to combine GPU stores into full-page programs. The buffer tracks both the data and the byte positions covered by each store. A page is flushed when it is full or when buffer pressure requires space. It is also flushed when the configured deadline expires, when a same-page read requires ordering, or during an idle tail drain. The default `strict` timeout policy records an incomplete-page error and does not label the operation as a full-page program. The opt-in `partial` compatibility policy pads and programs a partial page while recording both the partial program and padding bytes. The policy is selected with `hbf_write_timeout_policy`. Flushes are serialized per page. While a page MSHR is in flight, later writes remain buffered and cannot overwrite the entry. Erase-before-write and sequential in-block allocation follow the HBF profile. The controller returns the write response only after programming completes. This non-posted completion propagates write backpressure to the GPU store path.

We use the union of written byte positions to determine whether a page is complete (Figure 3). Rewriting an already covered region does not bring an incomplete page closer to full coverage. A full page can be programmed when its resources are available. An incomplete page forced to flush

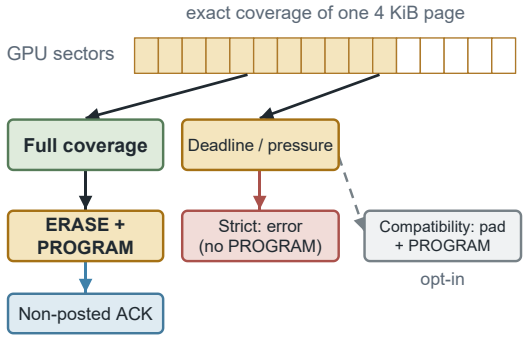



Figure 3: **Page-based write aggregation.** Incomplete pages trigger an error on flush or are padded in compatibility mode. The GPU receives a completion response only after the full page is written to NAND.

follows the strict-error or opt-in padding path. The acknowledgement follows PROGRAM because a GPU store completes only after the data has been programmed.

### 4.2.2 Page-grained Read Module

We use page-keyed MSHRs to combine outstanding reads to the same page. Sector or cache-line requests to one 4 KiB page can share one NAND read. In *demand mode*, the first request dispatches as soon as the page is ad-

mitted. Later requests to that page merge with the in-flight MSHR. In *aggregation mode*, the controller waits for a configured time window or byte threshold before issuing the page read. The aggregation wait is bounded by that configured window. Aggregation mode can reduce duplicate page reads for dense same-page traffic. Aggregation is an optional policy. The interface exposes these choices as `hbf_read_mode`, `hbf_read_agg_window`, and `hbf_read_agg_threshold`. The default aggregation threshold is one 4 KiB page.

Each read is scheduled on a subarray owned by the page's channel. The scheduler checks subarray availability and a configurable limit on active operations before issuing a page operation. Each sub-array implements READ, PROGRAM, and ERASE states with configurable timings. The default profile uses $t_R = 15\mu$s, $t_{PROG} = 200\mu$s, and $t_{BERS} = 2\,ms$. Each subarray has two NAND page buffers. A buffered-page reread pays the buffer-hit latency instead of a full media read when ordering permits. A logic-die page cache provides LRU reuse across kernels and access windows.

## 4.3 GPU–HBF Request Lifecycle

We connect these modules through the read and write sequences shown in Figure 4. Time advances downward in the diagram. The vertical spacing shows event order rather than measured durations. In panel (a), the initial read is mapped to a stack and tracked by page, so later same-page requests can join the outstanding operation. Channel waiting precedes READ or page-buffer service, after which data returns to the source. Panel (b) shows the additional write dependency: same-page stores first build complete coverage, then the page waits for service and is programmed, with erasure if needed. The write acknowledgement is returned only afterward.

Each request retains its source subpartition for return routing. The cube advances its controller, channel, and NAND state machines on the DRAM clock. Completed requests enter the source partition's return path and then follow the existing interconnect. Cache fills are handled by the ordinary L2 fill path when a corresponding cache miss is pending. DRAM requests continue through the existing DRAM path.

**Clock domains and cost.** The controller advances HBF media timers ($t_R$, $t_{PROG}$, $t_{BERS}$, page-buffer hits, and page-cache hits) with the DRAM clock used by the simulator. GPU-visible latency is converted to GPU core cycles when a request completes. Statistics retain both units and report the DRAM frequency and tick-to-$\mu$s conversion. The controller keeps one page-keyed MSHR map, one pending-page FIFO, one write-buffer map, and a set of busy subarrays. Only busy subarrays are advanced on each tick. The per-tick cost therefore follows the number of in-flight operations rather than the full array size. Scheduling scans the pending queue within the outstanding-request cap.

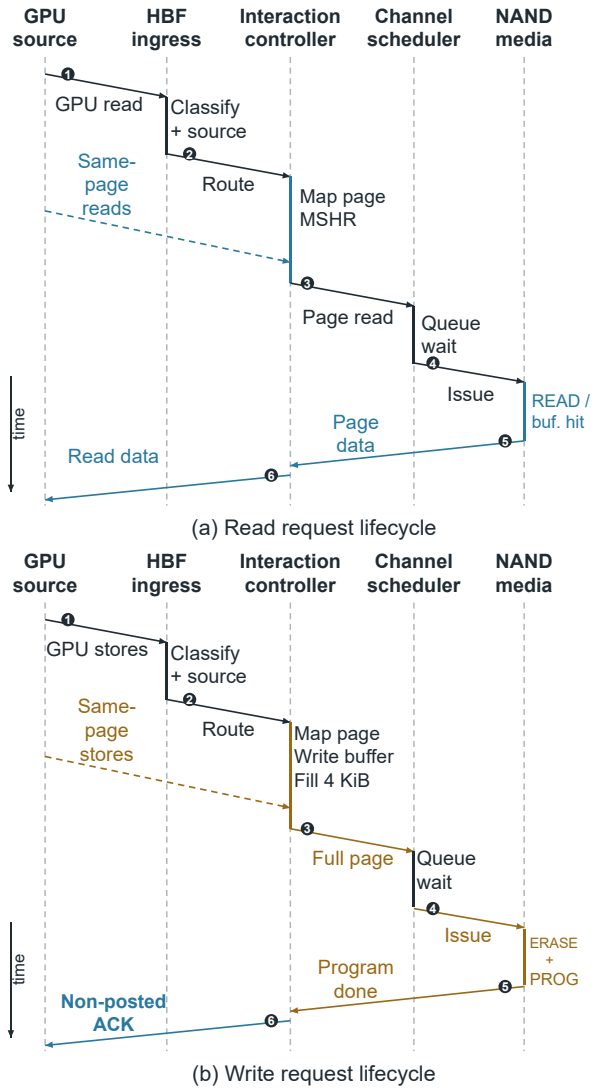



Figure 4: **Request lifecycle through HBF-Sim.** A GPU read or write request enters the interaction controller, is mapped to one stack and channel, and follows page tracking, queueing, and NAND service before read data or write status returns to the source GPU partition.

## 4.4 Trace Generation

We implement request tracing to connect GPU accesses with controller decisions and media operations. Each record identifies the request state and the work performed to serve it.

**State and accounting.** We record each request's identifier, operation, address, size, source, and arrival and completion times. Each request advances through INGRESS, MAPPED, optional write BUFFERED, QUEUED, MEDIAOP, RETURN, and COMPLETED states. GPU L2 and CUDA Shared Memory remain distinct from the HBF page cache and NAND page buffers. Bounded queues may delay a request but cannot change its owner. Device timers advance in DRAM-clock

ticks. GPU-visible latency is recorded in core cycles. Read amplification is *media bytes/requested bytes*, while write accounting records coverage, programs, padding, and ordering errors.

We expose configuration parameters for per-channel bandwidth, NAND timings, channel count and mapping, subarray count, page-cache and page-buffer sizes, write-buffer depth and deadline, placement, and media mode. Placement can be contiguous, page-interleaved, or selected by an explicit mapping table. Statistics include per-channel request and operation counts and link-stall ticks. They include first-request latency, aggregation wait, page-read count, requested bytes, media bytes, read amplification, and MSHR hits. They also include page-buffer and page-cache hits, partial-page programs, timeout flushes, idle drains, padding or strict-policy errors, read-after-write ordering, capacity utilization, and per-zone PEC spread.

The optional CSV trace records `sim_cycle`, `request_id`, `source_subpartition`, `op`, `address`, `page`, `channel`, `subarray`, `state`, and `bytes`. It also records `queue_depth`, `latency`, `cache_hit`, `mshr_hit`, and `error`. The trace file and trace detail are selected with `hbf_trace_file` and `hbf_trace_level`. The ordered 15-column schema is frozen as `hbf-trace-v1`. Validation rejects missing, added, or reordered columns. Full-system traces preserve GPU injection effects. Device-only replay can inject the same request stream without the GPU issue-rate limit.

## 4.5 Implementation

We implement HBF-Sim in C++ as an Accel-Sim extension and maintain the HBF source files in `hbf/`. The setup script applies the integration patch to a pinned simulator revision and copies the active cube, controller, media, and trace sources into the build tree. This keeps generated simulator files separate from the maintained HBF module.

We instantiate one logical HBF system with the configured number of independent stack controllers. Each stack owns its channels, controller queues, page cache, FTL, and trace writer. The system layer owns address routing and fair return arbitration. The controller stores page-keyed MSHRs and per-page write buffers. It instantiates concrete subarray state lazily, advances only the active-subarray set, and skips the HBF tick when the complete system is idle. The same sources build through the Make and CMake paths.

The artifact scripts compile the validation kernels, create timestamped configurations, and store binary and trace hashes beside each run. The trace remapper changes only global addresses and writes a manifest with the input and output hashes. The device replay tool consumes the same request schema as the simulator and is used to separate HBF service from GPU injection.

# 5 Platform Validation and Evaluation

We evaluate request handling, timing feedback, page service, and resource scaling. We first check functional behavior and compare fixed-arrival replay with coupled GPU execution. We then cross-check page service with MQSim, measure media scaling, and run a Qwen3 decode trace. Finally, we measure simulator cost and addressable capacity. Section 6 uses the same platform to evaluate placement and read/write isolation policies.

## 5.1 Methodology and Functional Validation

Unless otherwise specified, we use Accel-Sim with a 1.132 GHz GPU core clock and an 850 MHz DRAM clock. Scaling retains an 80-SM configuration. The placement and mixed-write studies use 8 shader clusters and 8 memory partitions. Default READ/PROGRAM/ERASE timings are 15/200/2000 $\mu$s. The placement diagnostic uses 20/40/80 DRAM ticks. These are model parameters rather than hardware measurements. Raw configurations, traces and binary hashes are retained.

We measure request latency from request creation to completion return by the HBF controller, including waiting before controller acceptance. We report the 95th percentile (p95) of read and write request latencies in the read/write interference study. GPU execution time covers the entire workload, including write completion. Both request latency and GPU execution time are measured in GPU clock cycles.

Our routing tests preserve all 64 request/completion pairs at 1–16 channels. The shared-staging probe produces 32 HBF requests, one page read and 31 MSHR hits. Demand and aggregation finish in 5,508 and 5,517 cycles. A remapped LUD trace preserves all 158 request/completion pairs. C++ coverage tests check repeated fragments and page boundaries. Successful strict-write traces require complete coverage before PROGRAM and acknowledgements after programming. These checks exercise different parts of the request path. Pairing request IDs with completion IDs detects lost or duplicate returns. The shared-staging probe checks that one page operation can serve multiple waiting requests. For writes, repeated fragments must not count as new byte coverage, and a complete page must be programmed before its acknowledgement is released. Our tests validate request handling in these workloads. Hardware calibration remains outside this evaluation.

## 5.2 MSHR Benefits and Timing Feedback

We test whether a fixed device-input trace preserves the benefit of page-keyed MSHR merging when admission pressure changes. The closed-loop configuration lets GPU execution and upstream queues respond to HBF completions. The open-loop configuration freezes controller-ingress times from a source run. Both use the same C++ HBF controller, media

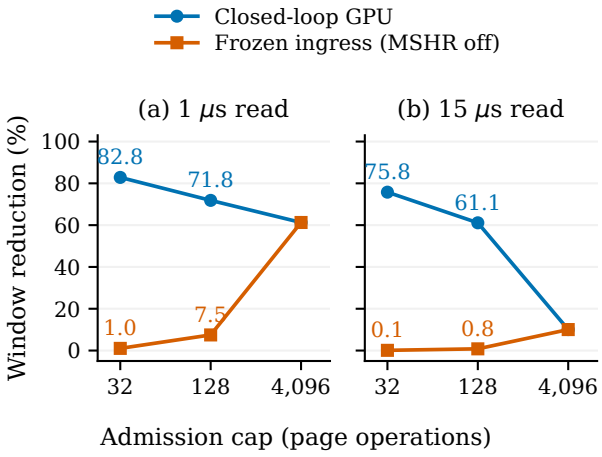



Figure 5: **Frozen ingress understates MSHR benefits under admission pressure.** HBF completion-window reduction relative to merging disabled, using 16 pages and 32 entries per page. Replay fixes the baseline's ingress times.

model, admission limits and return credits. The replay driver retains source request order and enforces at most one admission per memory partition per DRAM tick, but does not execute GPU cores.

We run 24 GPU configurations: 16 pages, 1 or 32 distinct 128 B entries per page, 32/128/4,096 admission slots, 1/15 $\mu$s reads, and merging on/off. Eight SMs and eight memory partitions use 1.132/0.850 GHz core/DRAM clocks. Four HBF channels, 32 subarrays and eight active slots are fixed. The logic-die cache is disabled and NAND page buffers are enabled. Every source replay reproduces its complete controller event trace exactly. We then replay each source into the opposite merging configuration, yielding 24 counterfactual runs with identical address/size multisets and conserved request/completion pairs.

We compare the window from first HBF admission to last HBF completion. This interval is available in both GPU simulation and device-only replay. Our evaluation shows that fixed-arrival replay understates the benefit of merging under admission pressure (Figure 5). We use the input trace captured with merging disabled as the replay baseline. At 15 $\mu$s and 128 slots, the closed loop reduces this window from 87,703 to 34,101 cycles (61.1%). Frozen ingress instead gives 86,987 cycles (0.8% reduction), a 60.3-percentage-point difference in predicted benefit. The full GPU kernel correspondingly falls from 93,699 to 40,164 cycles. With 32 slots, the window reductions are 75.8% and 0.1%. With 4,096 slots, both predict 10.0%. The 1 $\mu$s controls exhibit the same pressure-dependent pattern. All sparse-input controls and reverse transfers are retained in the artifact.

Our traces explain this difference through request admission. At 128 slots, enabling merging contracts the closed-loop ingress span from 86,972 to 414 cycles: fewer distinct page operations occupy admission slots, allowing upstream GPU requests to advance. Frozen ingress retains the source run's waiting even after merging is enabled. Conversely, disabling merging while replaying the merging-enabled trace changes the target window by only 1.8% at this point. The abstraction difference is directional. At 4,096 slots the source ingress schedules coincide and replay agrees. Our comparison shows that admission feedback affects the estimated benefit of controller policies under pressure.

| Channels | $t_R$ ($\mu$s) | HBF-Sim | MQSim | Difference |
|---|---|---|---|---|
| 1 | 1 | 64.032 | 85.126 | +32.9% |
| 4 | 1 | 16.080 | 21.316 | +32.6% |
| 1 | 15 | 960.032 | 981.126 | +2.2% |
| 4 | 15 | 240.080 | 245.316 | +2.2% |

Table 4: **External read-service comparison.** Burst completion windows in $\mu$s. Difference is $(T_{\mathrm{MQSim}}/T_{\mathrm{HBF}} - 1)$. The models retain their respective interface overheads. These differences are not errors against HBF silicon.

## 5.3 External Page-Service Cross-Check

We compare the HBF device driver with unmodified MQSim [32] (revision `51f0f2d`), restricting the comparison to their common read-page service. Eight paired configurations use 64 unique 4 KiB pages, one or four channels with one serial die/subarray per channel, and 1/15 $\mu$s reads. Page arrivals are either 50 $\mu$s apart or a burst spaced by 10 ns. Each logical page is one eight-sector NVMe request in MQSim and 32 merged 128 B requests in HBF-Sim. Both service 256 KiB. HBF-Sim performs exactly 64 page operations. Data/page caches are disabled. MQSim uses ideal address mapping and no preconditioning. The read-only run avoids GC and write-management differences.

Nominal channel bandwidth is 192 GB/s in both models. MQSim uses a synthetic 192-byte channel at 1,000 MT/s and four 192 GB/s PCIe lanes to keep host bandwidth from dominating. These are comparison parameters, not a projected SSD. Native NVMe/ONFI processing remains in MQSim. HBF-Sim retains its bandwidth-credit link and GPU-sized fragments. We therefore compare logical arrival-to-completion windows, without equating the two device interfaces.

Our comparison shows similar scaling in both models, with an offset caused by their different interface models (Table 4). Increasing $t_R$ from 1 to 15 $\mu$s adds 896 $\mu$s with one channel and 224 $\mu$s with four in both models, matching $64\Delta t_R/C$. MQSim retains, for example, a 290 ns single-page ONFI command phase in this configuration. Its burst windows exceed HBF-Sim's by 2.2% at 15 $\mu$s and about 33% at 1 $\mu$s. The isolated controls also retain an interface offset. Their long arrival gaps are excluded from claims of close agreement. This external check supports the implemented read-timing and parallel-service trends while exposing the scope of our link abstraction. It does not calibrate HBF silicon, GPU timing, or write behavior.

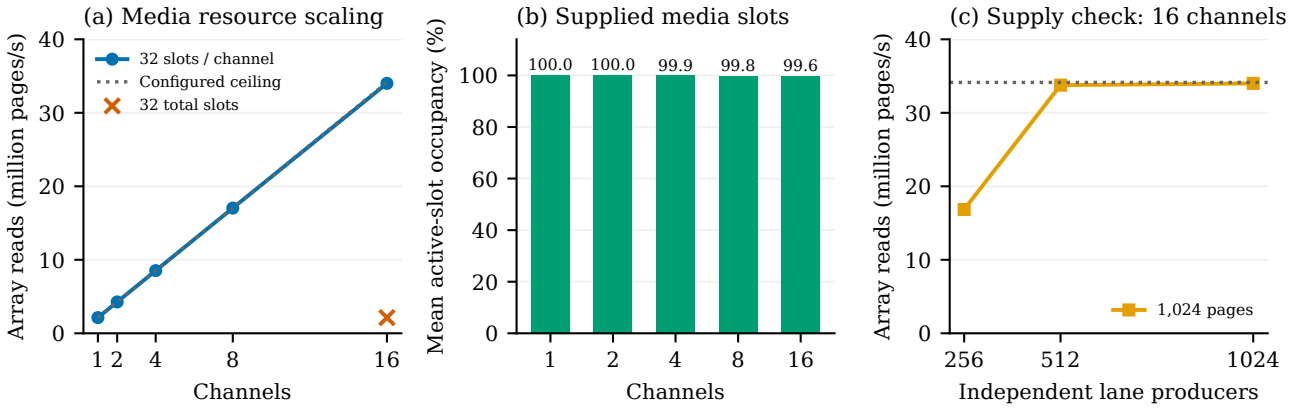



Figure 6: **Media scaling with 32 subarrays and active slots per channel at 15 $\mu$s reads.** Throughput counts array pages, not GPU payload bytes. Occupancy and producer-count controls check saturation.

## 5.4 Media Resource Scaling

We test media scaling with 1,024 lanes, each reading a different page to provide independent requests. Each 4 B load generates a 32 B sector request and one 4 KiB array read. Both cache shortcuts are disabled. We retain GPU backpressure and scale from 1 to 16 channels, with 32 subarrays and active slots per channel, 4,096 admission entries and default media timings.

Our evaluation shows that array-service bandwidth increases from 8.738 to 139.285 GB/s (15.94×), with 99.6–99.99% media occupancy (Figure 6). At sixteen channels, 256 producers fill only half of the 512 slots. 512/1,024 producers deliver 138.291/139.285 GB/s. Doubling the stream to 2,048 pages/producers changes throughput by about 0.2%, while fixing total slots at 32 restores 8.738 GB/s. Array bandwidth counts internal page reads: the corresponding sixteen-channel GPU sector bandwidth is only 1.088 GB/s. Our fixed-slot control shows that the gain comes from additional media parallelism. Increasing the channel count alone does not improve throughput when the total number of active slots remains fixed. The producer-count controls check that the main sweep has enough outstanding work to occupy the added resources.

## 5.5 LLM Decode Workload

We use Qwen3-1.7B [26] to exercise the GPU–HBF path with a real pretrained language model. The standard Transformers implementation runs in BF16 with eager attention, batch size one, and thinking disabled. The model has 28 layers, hidden dimension 2,048, 16 query heads and eight KV heads. Native inference on an RTX 4090 processes a 27-token prompt and produces eight tokens using greedy decoding. We use Accel-Sim's NVBit-based tracer [33] to capture the second decode forward pass after prefill, with a 29-token context, preserving all 1,819 GPU kernels across the full model. The native run supplies the input token and KV state for this captured step.

The trace remapper translates only the recorded parameter allocations into the HBF address range, preserving offsets and shared-weight aliases. KV, activations and library scratch remain on the ordinary GPU-memory path. Replay uses an SM80 timing profile with eight SMs, eight memory partitions, and 1.410/1.512 GHz core/DRAM clocks. One HBF stack provides 16 channels, 512 total subarrays, 512 active-operation slots and 4,096 admission entries. READ/PROGRAM/ERASE timings are 15/200/2000 $\mu$s. MSHR merging and NAND page buffers are enabled. The logic-die page cache is disabled. This configuration is evaluated separately from the mechanism studies.

| Metric | Result |
|---|---|
| Captured / completed GPU kernels | 1,819 / 1,819 |
| GPU kernel-sequence time (ms) | 249.95 |
| HBF read requests | 107,540,992 |
| HBF controller read completions | 107,540,992 |
| Returned sector bytes | 3.441 GB |
| Array-read bytes | 3.441 GB |

Table 5: **Qwen3-1.7B decode execution and HBF traffic.** Values are taken from final simulator counters. Time covers the GPU kernel sequence. Array bytes are page services minus NAND page-buffer hits, multiplied by 4,096 bytes. GB denotes $10^9$ bytes.

Our simulator completes all 1,819 captured kernels. We report the final execution and traffic counters in Table 5. The sequence takes 352,434,530 GPU cycles (249.95 ms). The controller records 107,540,992 read requests and 107,540,992 read completions.

This experiment demonstrates HBF-Sim's capability to execute the complete GPU-kernel sequence of a captured Qwen3-1.7B decode step. Starting from cold simulated caches, the reported execution time focuses directly on the memory-bandwidth-bound GPU kernel execution during active token generation, with model weights residing in HBF while KV caches and activations remain in GPU memory.

## 5.6 Simulator Cost and Capacity

We measure simulator cost with five alternating idle-path repetitions per mode. The median host times are 6.768/6.964 s and peak RSS of 58,832/59,016 KiB with HBF disabled/enabled. Both execute 181,707 cumulative cycles over 32 kernels. We also test a four-stack, 512 GiB address space with a sparse probe. It completes 20 sectors across five boundary/interior pages without mapping errors or eager metadata allocation. It tests addressing and capacity accounting, not full-capacity sustained performance.

# 6 HBF-Sim-based Optimization Cases

Having checked the mechanisms in Section 5, we use HBF-Sim to study two policy questions: how page placement affects read service and execution, and when resource isolation helps protect reads from writes. These cases compare static policies through the mapping and scheduling interfaces while retaining the GPU request/completion path. They use controlled GPU workloads. Their findings concern the tested access patterns and resource configurations.

## 6.1 Locality/Parallelism-Aware Placement

Page placement can reduce repeated page service and lower read-side page-service amplification. We first measure this benefit and its effect on kernel execution, then examine how placement affects subarray parallelism.

**Policies and setup.** We compare page interleave, two-page grouping, and contiguous placement through the address-mapping interface in Section 4. Figure 7 shows their channel assignments. Interleave rotates channels after each page, while grouped placement rotates after every two pages. Contiguous placement divides the configured capacity into channel-owned regions. Each mapping determines both the channel and subarray of a page and remains fixed throughout a run.

The study covers nine configurations: three mappings and 1/8/32 distinct 128 B entries per page, with shortened read timing ($t_R = 20$ DRAM ticks). We use a CUDA read microbenchmark with 16 pages, four channels, 32 total subarrays, eight active slots, and an admission cap of 256. MSHR merging and NAND page buffers are enabled; the logic-die page cache is disabled.

**Reducing page-service amplification.** With $B$ denoting requested read bytes, page-service amplification and array-read amplification are:

$$A_s = \frac{4096 N_{\text{service}}}{B}, \qquad A_a = \frac{4096 N_{\text{array}}}{B}.$$

$N_{\text{service}}$ counts page services, including page-buffer hits, while $N_{\text{array}}$ counts NAND array reads. Figure 8 shows that placement reduces page-service amplification as accesses become

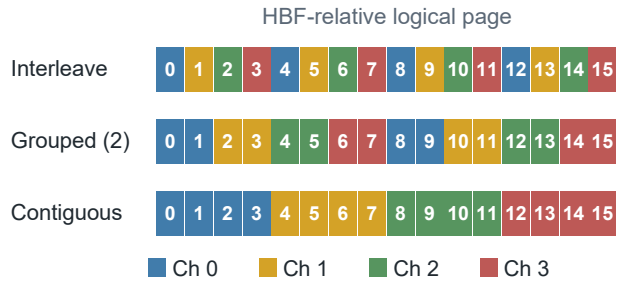



Figure 7: **Channel assignments under three placement policies.** Interleave rotates channels after each page; grouped placement rotates after every two pages. Contiguous placement assigns consecutive active pages to each channel.

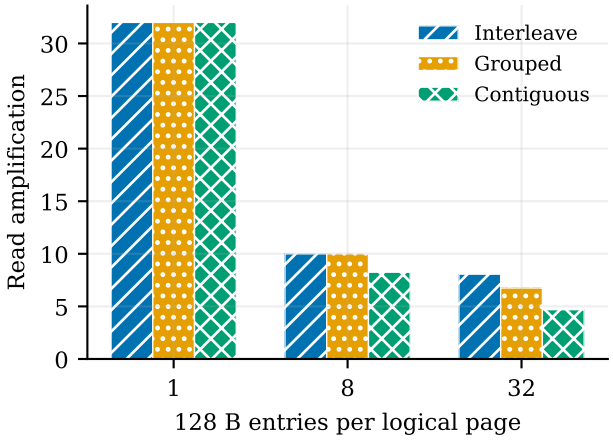



Figure 8: **Page-service amplification under shortened timing ($t_R = 20$ DRAM ticks).** At 32 entries per page, contiguous placement reduces page-service amplification by 41.9% relative to interleave. Page services include NAND page-buffer hits.

denser within each page. At 32 entries per page, interleave, grouped, and contiguous placement require 129, 108, and 75 page services, respectively. Contiguous placement lowers amplification from 8.0625 to 4.6875 relative to interleave, a 41.9% reduction.

**Effect on kernel execution.** All three mappings perform 16 array reads at 32 entries per page; the remaining page services are served by the NAND page buffers. Thus, the reduction in page-service amplification leaves array work unchanged. Kernel time rises slightly from 6,505 cycles with interleave to 6,550 cycles with contiguous placement, an increase of 0.69%. The next experiment examines how the distribution of accesses across subarrays affects execution time.

**Resource conflicts and parallelism.** We use a separate 36-run study with default read timing ($t_R = 15\,\mu$s), 64/128 active pages, 1/8 entries per page, and logical-page strides of 1/4/32. We compare built-in interleave, an explicit modulo-channel map, and active-page striping. For active pages $p_i = iD$, where

| Pages | Stride | Subarrays | Kernel speedup | |
|---|---|---|---|---|
| | | I / S | $E = 1$ | $E = 8$ |
| 64 | 1 | 32 / 8 | 1.00 | 1.00 |
| 64 | 4 | 8 / 4 | 0.51 | 0.51 |
| 64 | 32 | 1 / 4 | 3.94 | 3.93 |
| 128 | 1 | 32 / 8 | 1.06 | 1.00 |
| 128 | 4 | 8 / 4 | 0.51 | 0.51 |
| 128 | 32 | 1 / 4 | 3.97 | 3.97 |

Table 6: **Active-page striping (S) versus built-in interleave (I) under default timing.** Speedup is kernel cycles of I divided by S. Values below one indicate a slowdown. $E$ is the number of 128 B entries per page. Subarray counts are distinct resources used over a run, with identical counts for both $E$ values. The concurrent-operation cap is eight.

$D$ is the page stride and $C$ is the channel count, explicit modulo assigns channel $p_i \bmod C$ and active-page striping assigns $i \bmod C$. The resource budget remains four channels, 32 total subarrays, and eight active slots, with an admission cap of 4,096. The two studies use different simulator binaries; policy comparisons are made within each study.

The mappings also differ in their subarray assignments. For logical page $p$, built-in interleave uses $\lfloor p/C \rfloor$ as the channel-local page index, while explicit maps use $p$. The local subarray index is this value modulo the number of subarrays in the channel. The explicit modulo map therefore provides a control with the same channel assignments as interleave but a different subarray distribution.

For 64 pages, one entry per page and stride 32, active-page striping reduces kernel time from 1,092,373 to 277,353 cycles (3.94× speedup), with 64 array reads in both runs. READ traces show that striping spreads accesses over four subarrays instead of one, allowing more reads to proceed in parallel. At stride 4, the same policy change increases time from 141,530 to 277,353 cycles. Although requests become balanced across channels, they use four subarrays instead of eight, reducing the available parallelism. Table 6 shows the same trend across both working-set sizes and entry counts. With stride 1, both mappings use at least eight subarrays, enough to reach the eight-operation cap, and their execution times are similar.

**Design implication.** Page placement can lower read-side page-service amplification. Its effect on execution time also depends on array work and the parallelism available across subarrays. Placement policies should therefore consider page-service reuse together with channel and subarray assignments. HBF-Sim's mapping interface and request traces make both effects visible when evaluating static placement policies.

## 6.2 Protecting Reads from Write Interference

**Question and policies-2:** *When reads arrive during a long flash program, does read-priority scheduling provide adequate protection, or should reads receive separate resources?* We compare shared FCFS, shared read priority, shared write drain, and static 1R/3W, 2R/2W and 3R/1W channel partitions using read priority within the partitioned configuration. Scheduling changes candidate selection. Partitioning changes the page-to-channel map. Neither interrupts an issued PROGRAM. Read and write pages are disjoint, and all configurations retain strict full-page writes and acknowledgements after programming.

**Matched experiment.** The phase study holds four channels, four total subarrays and four active slots fixed. It varies writes of 4/16 pages and the nominal read-start delay, with matched read-only controls. Of 37 configurations, 36 complete and one violates the strict page-assembly deadline. The main deadline is 10,000 DRAM ticks (11.76 $\mu$s). The matched extended-deadline control uses 400 $\mu$s. The read stream contains 512 sector requests, and each written page contributes 128 sectors with complete byte coverage. We report creation-to-controller-return latencies for reads and writes, together with whole-kernel completion time.

**Read benefit and write cost.** With four written pages and a 100 $\mu$s read-start delay, 3R/1W reduces read p95 from 232,456 to 169,730 GPU cycles relative to shared read priority (27.0%). Kernel time increases from 776,097 to 911,850 cycles (17.5%), while write p95 increases from 226,715 to 905,889 cycles (4.00×). The reserved read resources reduce exposure to programming, while the single write channel limits write parallelism.

The benefit changes with arrival phase (Figure 9). At zero nominal delay, 3R/1W increases read p95 from 118,999 to 152,968 cycles. At 50 $\mu$s it lowers p95 from 289,058 to 169,710 cycles. At 250 $\mu$s, shared read priority again has lower read p95. Priority can reorder pending work, but an already-issued program leaves residual blocking. Nominal launch delay and actual program overlap are therefore evaluated separately using request and media events.

**Design implication.** HBF channel allocation should use read/write isolation selectively, balancing read-tail latency targets against write parallelism and whole-workload completion time. For an illustrative budget of read p95 at most 1.5× the matched read-only p95, the four-page sweep gives a concrete choice between shared read priority and 3R/1W. At 0, 150 and 250 $\mu$s, sharing satisfies the budget and finishes the kernel sooner. At 50 and 100 $\mu$s, only 3R/1W of these two options meets the budget. The 16-page deadline failure further shows why strict-write feasibility must be checked before ranking performance.

# 7 Discussion

HBF-Sim models page mapping, read coalescing, write aggregation, channel contention, and completion feedback to GPU execution. ECC, read retry, refresh, and thermal effects are not modeled in the current version. These mechanisms may affect tail latency and sustained bandwidth in real de-

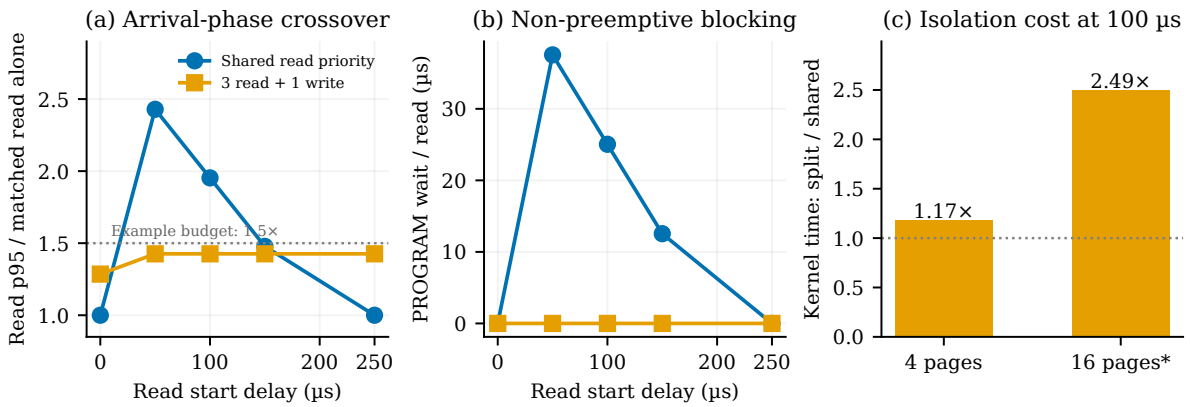



Figure 9: **Read protection depends on arrival phase and resource allocation.** (a) Read p95 is normalized to a matched read-only run. (b) Average time per read spent queued while its target subarray is executing PROGRAM. The average includes all read requests and is not added to kernel time. (c) Isolation changes whole-kernel time. The starred 16-page comparison uses a matched, extended 400 $\mu$s assembly deadline. The original isolated configuration is infeasible under the shorter deadline. The 1.5× line is an illustrative budget.

vices, but they do not invalidate the functionally consistent request/completion behavior or the semantically correct page-level interactions captured by our simulator. Since HBF hardware is unavailable, we check selected properties from the OCP specification [24] without hardware-calibrated measurements: requests access only channel-local resources, programming requires a complete 4 KiB page, and write acknowledgements follow programming (Section 5.1). We also perform timing experiments to examine page-service and resource-scaling trends. These evaluations demonstrate that HBF-Sim faithfully preserves the OCP-specified request/completion semantics, though they do not establish cycle-level accuracy against physical HBF silicon.

Future work can extend the static policies evaluated here to adaptive placement and channel allocation based on access patterns. Transaction-level AXI/UCIe modeling would allow closer examination of protocol behavior beyond the current bandwidth-and-credit model. Hardware measurements, when available, could calibrate timing parameters and test whether the observed trade-offs hold on real devices.

## 8 Related Work

**Device and interconnect models.** Xerxes [2] studies scalable CXL fabrics, CXL-MQSim [35] studies CXL-enabled SSDs, and WARP [31] combines device emulation with real-device characterization. SwarmIO [14] emulates SSDs for GPU-initiated I/O. Sano et al. [28] use FPGA prototypes to study GPU graph processing with microsecond-latency CXL memory. FlashSim [17] and NANDFlashSim [10] model flash timing and media operations. Ramulator [16] and DRAMsim3 [20] model DRAM devices. These models address complementary device or interconnect questions. Their interfaces and media-management assumptions require examination when reused for an HBF tier.

**HBF architecture and systems studies.** $H^3$ [7] explores hybrid HBM–HBF inference, and HAVEN [8] targets approximate nearest-neighbor search. TileLens [9] studies memory layout and prefetching for large-granularity memory. FlashAccel [34] co-designs HBF hardware and software for LLM inference. FLINT [23] studies HBF-based inference and read coalescing. The full-stack characterization [21] and IEEE CAL study [30] examine HBF deployment trade-offs. Petrucci et al. [25] compare HBF and LPDDR capacity tiers using a roofline model. These architecture and application studies motivate reusable evaluation infrastructure. While these studies demonstrate HBF's potential in specific workloads, their models lack closed-loop timing feedback between GPU execution pipelines and HBF media constraints.

## 9 Conclusion

This paper addresses the lack of an open and faithful simulation platform for exploring high-bandwidth flash in next-generation GPU big-memory systems. We present **HBF-Sim**, an extensible Accel-Sim-integrated framework that models the full GPU–HBF request/completion path for systematic design exploration. Our results show that HBF-Sim is functionally correct end to end, captures closed-loop effects that simpler models miss, and reproduces expected page-service and resource-scaling trends. Using HBF-Sim, we further evaluate the impact of page placement and resource isolation on performance, and identify two design implications for HBF systems. HBF-Sim thus provides an open foundation for reproducible HBF research before production hardware becomes available.

## References


[1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. arXiv:2312.11514, 2023.

[2] Yuda An, Shushu Yi, Bo Mao, Qiao Li, Mingzhe Zhang, Diyu Zhou, Ke Zhou, Nong Xiao, Guangyu Sun, Yingwei Luo, and Jie Zhang. Xerxes: Extensive exploration of scalable hardware systems with CXL-based simulation framework. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*, pages 329–345. USENIX Association, 2026.

[3] Ali Bakhoda, George L. Yuan, Wilson W. L. Fung, Henry Wong, and Tor M. Aamodt. Analyzing CUDA workloads using a detailed GPU simulator. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009.

[4] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011.

[5] CXL Consortium. Compute express link (CXL) specification 3.1, 2023.

[6] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. AI and Memory Wall. *IEEE Micro*, 44(3):33–39, 2024.

[7] Minho Ha, Euiseok Kim, and Hoshik Kim. $H^3$: Hybrid architecture using high bandwidth memory and high bandwidth flash for cost-efficient LLM inference. *IEEE Computer Architecture Letters*, 25(1):49–52, 2026.

[8] Po-Kai Hsu, Weihong Xu, Qunyou Liu, Tajana Rosing, and Shimeng Yu. HAVEN: High-bandwidth flash augmented vector engine for large-scale approximate nearest-neighbor search acceleration. arXiv:2603.01175, https://arxiv.org/abs/2603.01175, 2026.

[9] Jae Hyung Ju, Euijun Chung, Hritvik Taneja, Anish Saxena, Shinnung Jeong, Hyesoon Kim, and Moinuddin K. Qureshi. Tilelens: Efficiently using large-granularity memory systems with transparent two-dimensional memory layout. arXiv:2607.04031, https://arxiv.org/abs/2607.04031, 2026.

[10] Myoungsoo Jung, Ellis H. Wilson, David Donofrio, John Shalf, and Mahmut T. Kandemir. NANDFlashSim: Intrinsic latency variation aware NAND flash memory system modeling and simulation at microarchitecture level. In *Proceedings of IEEE MSST*, 2012.

[11] Myoungsoo Jung, Jie Zhang, Ahmed Abulila, Miryeong Kwon, Narges Shahidi, John Shalf, Nam Sung Kim, and Mahmut Kandemir. SimpleSSD: Modeling solid state drives for holistic system simulation. *IEEE Computer Architecture Letters*, 17(1):37–41, 2018.

[12] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y. Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-Throughput LLM Inference with Shared Prefixes. arXiv:2402.05099, https://arxiv.org/abs/2402.05099, 2024.

[13] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. Accel-sim: An extensible simulation framework for validated GPU modeling. In *Proceedings of the 47th International Symposium on Computer Architecture (ISCA)*, 2020.

[14] Hyeseong Kim, Gwangoo Yeo, and Minsoo Rhu. SwarmIO: Towards 100 Million IOPS SSD Emulation for Next-generation GPU-centric Storage Systems. arXiv:2604.06668, https://arxiv.org/abs/2604.06668, 2026.

[15] Seeyeon Kim, Juhyeong Jin, and Joo-Young Kim. Beyond capacity: Scalable moe llm inference via high-bandwidth flash with direct gpu and hbm paths, 2026.

[16] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer Architecture Letters*, 15(1):45–49, 2016.

[17] Youngjae Kim, Brendan Tauras, Aayush Gupta, and Bhuvan Urgaonkar. FlashSim: A simulator for NAND flash-based solid-state drives. In *Proceedings of the 1st International Conference on Advances in System Simulation (SIMUL)*, pages 125–131, 2009.

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023.

[19] Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Matias Bjørling, and Haryadi S. Gunawi. The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator. In *Proceedings of*

*the 16th USENIX Conference on File and Storage Technologies (FAST)*, pages 83–90. USENIX Association, 2018.

[20] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.

[21] Zhuoran Li, Zhuohang Bian, Xin Huang, Yibo Zhao, Guangyu Sun, and Youwei Zhuo. HBF Sucks? a full-stack characterization of high-bandwidth flash for kv-centric LLM serving. arXiv:2608.11668, https://arxiv.org/abs/2608.11668, 2026.

[22] Micron Technology. Micron 9550 SSD Series Technical Product Specification. https://www.micron.com/content/dam/micron/global/public/products/data-sheet/ssd/9550-nvme-ssd-tech-prod-spec.pdf, February 2025. Revision A, February 2025.

[23] Geraldo F. Oliveira, Arash Tavakkol, Xiangyu Zhu, Ahmet Caner Yüzügüler, Vamanan Arulchelvan, Lukas Cavigelli, Renzo Andri, Mohammad Sadrosadati, Jia Xinglei, Onur Mutlu, Zhou Ke, Shai Bergman, and Ji Zhang. FLINT: Efficiently leveraging high bandwidth flash for capacity-scalable LLM inference acceleration. arXiv:2608.25062, https://arxiv.org/abs/2608.25062, 2026.

[24] Open Compute Project. OCP High Bandwidth Flash (HBF) High-Level Base Die Specification, Version 0.7.0. Open Compute Project specification page, 2026. SK hynix and SanDisk.

[25] Vinicius Petrucci, Felippe Zacarias, and Vishal Tanna. Is High-Bandwidth Flash All You Need? In *3rd Workshop on Hot Topics in System Infrastructure (HotInfra)*, 2026.

[26] Qwen Team. Qwen3-1.7B: Model card. https://huggingface.co/Qwen/Qwen3-1.7B. Accessed September 15, 2026.

[27] Samsung Electronics. Samsung NVMe SSD 983 DCT: Product Brief. https://semiconductor.samsung.com/resources/brochure/Data_Center_SSD_983_DCT.Product_Brief.pdf, July 2018. July 2018 edition.

[28] Shintaro Sano, Yosuke Bando, Kazuhiro Hiwada, Hirotsugu Kajihara, Tomoya Suzuki, Yu Nakanishi, Daisuke Taki, Akiyuki Kaneko, and Tatsuo Shiozawa. GPU Graph Processing on CXL-Based Microsecond-Latency External Memory. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 962–972. ACM, 2023.

[29] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of ICML*, 2023.

[30] Dowon Son, Yonggon Park, Hyunuk Cho, Hyungkyu Ham, Onur Mutlu, Sungjin Lee, Gwangsun Kim, and Jisung Park. Exploring high-bandwidth flash for modern LLM inference: Opportunities and challenges. *IEEE Computer Architecture Letters*, 25(2):251–254, 2026. arXiv:2608.13868.

[31] Inho Song, Shoaib Asif Qazi, Javier Gonzalez, Matias Bjørling, Sam H. Noh, and Huaicheng Li. Characterizing and emulating FDP SSDs with WARP. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*, pages 347–362. USENIX Association, 2026.

[32] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim: A framework for enabling realistic studies of modern multi-queue SSD devices. In *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST)*, pages 49–66. USENIX Association, 2018.

[33] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. NVBit: A Dynamic Binary Instrumentation Framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 372–383. ACM, 2019.

[34] Xinyu Wang, Yalong Xue, Xiaotian Sun, Xiaoyu Zhang, Xinjiang Zhang, Chunmeng Dou, Xueqi Li, and Xiaoming Chen. Flashaccel: Leveraging high-bandwidth flash (HBF) for high-throughput LLM inference. arXiv:2607.10186, https://arxiv.org/abs/2607.10186, 2026.

[35] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhyung Park, Jin-yong Choi, Eyee Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S. Kim. Overcoming the memory wall with CXL-enabled SSDs. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 601–617. USENIX Association, 2023.

[36] Dongha Yoon, Hansen Idden, Jinshu Liu, Berkay Inceisci, Sam H. Noh, and Huaicheng Li. Cylon: Fast and accurate full-system emulation of CXL-SSDs. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*, pages 313–327. USENIX Association, 2026.